DBMS CAPSTONE PROJECT REPORT

[ENCS204]

ADAPTIVE LEARNING PLATFORM

Submitted in partial fulfilment of the requirement of the degree of

BACHELORS OF TECHNOLOGY

in

CSE CORE (Section - A)

to

K.R Mangalam University

by

Nakul (2401010025)

Under the supervision of

MS. MANSI KAJAL

Department of Computer Science and Engineering
School of Engineering and Technology
K.R Mangalam University, Gurugram - 122001, India

January 2026

INDEX

S.NO.	TOPIC	Page NO.
	Abstract	3
1.	Introduction	4
2.	Motivation	5
3.	Literature Review	6
4.	Gap Analysis	7
5.	Problem Statement	8
6.	Objectives	9
7.	Tools / Technologies Used	10
8.	System Architecture	11-12
9.	Methodology	13
10.	Conclusion	14
11.	References	15

ABSTRACT

The Adaptive Learning Platform is a comprehensive full-stack web application developed to deliver a personalised and intelligent online learning experience. The system is built using HTML, CSS, and JavaScript on the frontend, Python FastAPI as the backend framework, and MySQL as the relational database, ensuring efficient data handling and smooth interaction between system components.

Traditional e-learning platforms deliver the same content to every student regardless of their knowledge level, learning pace, or performance history. This one-size-fits-all approach leads to disengagement for advanced learners and frustration for beginners. Moreover, educators lack real-time insights into individual student progress, making it difficult to provide targeted support.

This project eliminates these drawbacks by providing a fully automated and intelligent adaptive learning engine that continuously evaluates each student's skill level based on their quiz performance and adjusts content difficulty accordingly. The system incorporates a Bayesian-inspired Knowledge Tracing algorithm to model learner knowledge and update skill estimates after every quiz attempt.

Key features of the platform include role-based access control for students, instructors, and administrators; an adaptive quiz engine that selects questions of appropriate difficulty; a gamification system with XP points, daily streaks, and achievement badges; a real-time progress analytics dashboard; and a smart lesson recommendation engine that prioritises weak topic areas for review.

The platform serves as an effective solution for modern educational institutions and online course providers, demonstrating how web technologies and database management systems can be combined to build scalable, intelligent, and student-centred learning applications.

KEYWORDS: Adaptive Learning, FastAPI, MySQL, Knowledge Tracing, Gamification, Quiz Engine, Progress Analytics, Role-Based Access Control

1. INTRODUCTION

In today's digital era, the demand for personalised and technology-driven education has significantly increased. Online learning platforms have become a popular choice for students and professionals seeking flexible, self-paced education. However, managing such platforms while delivering a truly adaptive experience poses several technical and pedagogical challenges.

Traditional e-learning systems deliver uniform content to all learners irrespective of their individual skill level, learning speed, or performance history. This approach fails to address the diverse needs of students — advanced learners find the content too easy and disengage, while beginners struggle with content that moves too quickly.

To address these challenges, the Adaptive Learning Platform is developed as a full-stack web application that intelligently adjusts learning content and quiz difficulty based on each student's assessed skill level. The system integrates user management, course delivery, adaptive quizzes, progress tracking, and gamification into a single cohesive platform.

The system is designed with a clean, modern user interface that allows students to browse and enrol in courses, consume lesson content, and take adaptive quizzes. Instructors can create and manage course content through a dedicated admin panel. Administrators have full access to all platform features.

By replacing static content delivery with an adaptive, data-driven approach, the application improves student engagement, accelerates learning outcomes, and provides educators with actionable insights into individual and cohort performance.

2. MOTIVATION

The motivation behind developing the Adaptive Learning Platform is to overcome the fundamental limitations of traditional e-learning systems and provide a modern, intelligent, and student-centred solution.

Static learning platforms treat all students identically, ignoring the fact that every learner has a unique pace, background, and set of strengths and weaknesses. As the volume of learners grows, the inability to personalise content leads to poor outcomes and high dropout rates.

Key Motivating Factors:

- Inefficiency of static content delivery — All students receive the same material regardless of prior knowledge or current skill level.
- Lack of real-time performance feedback — Educators cannot track individual progress or identify struggling students in time to intervene.
- Poor student engagement — Without personalisation, students quickly lose motivation, especially when content is too easy or too difficult.
- Absence of gamification — Modern learners respond positively to reward systems; traditional platforms lack XP, badges, and streak mechanics.
- Need for intelligent recommendations — Students benefit greatly from being guided to their weakest topics rather than following a rigid linear path.
- Database-driven personalisation — Modern DBMS capabilities allow real-time skill tracking and adaptive content selection that were not previously possible.

The proposed system is designed to address all these issues by providing a Bayesian-inspired adaptive engine, a gamified reward system, and a comprehensive analytics dashboard.

3. LITERATURE REVIEW

A review of existing adaptive learning systems and e-learning platforms reveals both the progress made and the significant gaps that remain in the field.

Early e-learning systems such as Learning Management Systems (LMS) like Moodle and Blackboard provided structured course delivery but lacked any adaptive capability. All students followed the same learning path regardless of performance or prior knowledge.

Subsequent research into Intelligent Tutoring Systems (ITS) introduced the concept of learner modelling, where a system maintains a model of the student's knowledge state. Bayesian Knowledge Tracing (BKT), introduced by Corbett and Anderson (1994), became one of the most influential approaches, modelling the probability that a student has mastered a concept based on their response history.

More recently, platforms such as Khan Academy and Duolingo have demonstrated the effectiveness of adaptive learning at scale, using performance data to recommend next steps and adjust difficulty. However, these are proprietary systems that do not expose their underlying architecture for academic study or replication.

Observations from Existing Systems:

- Most LMS platforms lack genuine adaptive capability — content paths are fixed by the instructor
- Gamification features such as XP and streaks significantly improve learner retention and engagement
- Real-time progress analytics are rarely available to both students and instructors simultaneously
- Open-source adaptive platforms often lack polished UI and are difficult to deploy
- Integration of role-based access control with adaptive features is rarely addressed in academic projects

This project builds upon these findings by implementing a lightweight but effective Bayesian-inspired skill tracking engine, combined with a full-featured web application that is deployable on standard infrastructure using FastAPI and MySQL.

4. GAP ANALYSIS

The analysis of existing e-learning systems reveals several critical gaps that the Adaptive Learning Platform is designed to address:

Gaps in Existing Systems:

- Lack of genuine adaptive content delivery based on real-time skill assessment
- Absence of a dynamic quiz engine that adjusts difficulty per learner
- No integration of gamification mechanics such as XP, streaks, and badges
- Limited or no smart recommendation of next lessons based on weak topic identification
- Insufficient role-based access control separating student, instructor, and admin capabilities
- Poor database design leading to slow queries and inability to support concurrent users
- No centralised learner profile tracking skill level, learning speed, and topic mastery

How This Project Bridges the Gaps:

- Implements a Bayesian-inspired skill level update algorithm after every quiz attempt
- Quiz engine selects questions at difficulty 1 (easy), 2 (medium), or 3 (hard) based on learner skill score
- Full gamification system with XP rewards per lesson, daily streak tracking, and five achievement badges
- Smart recommendation engine prioritises lessons from identified weak topic areas
- Three-role access system (student, instructor, admin) with JWT-based authentication
- Normalised MySQL schema with foreign key constraints ensuring data integrity
- Centralised learner_profiles table storing skill level, XP, streak, and topic vectors per user

5. PROBLEM STATEMENT

Traditional online learning platforms suffer from a fundamental design flaw: they deliver identical content to every student regardless of their current knowledge state, learning history, or performance on assessments. This static approach fails to meet the diverse needs of modern learners and results in suboptimal learning outcomes.

In existing systems, students who already understand a topic are forced to consume beginner-level content, while struggling students receive advanced material without adequate preparation. Neither group receives the targeted support they need to progress efficiently.

Additionally, quiz systems in traditional platforms assign the same questions to all students, making it impossible to distinguish between learners at different stages of mastery. Without dynamic difficulty adjustment, quizzes fail to accurately measure or develop a student's true capability.

From an instructor perspective, existing platforms provide limited real-time insight into individual student performance. Educators cannot easily identify which students are struggling, which topics are poorly understood at the cohort level, or how to prioritise their interventions.

Therefore, there is a clear need for a system that can:

- Continuously assess and update each student's skill level based on quiz performance
- Dynamically select quiz questions of appropriate difficulty for each individual learner
- Identify weak topic areas and proactively recommend relevant lessons
- Provide students with motivating feedback through gamification mechanics
- Give instructors and administrators actionable analytics on student progress
- Maintain a secure, scalable database backend capable of supporting multiple concurrent users

The Adaptive Learning Platform is proposed as a solution to these problems, delivering an intelligent, personalised, and engaging learning experience through the integration of adaptive algorithms, gamification, and a robust database-driven architecture.

6. OBJECTIVES

The primary objective of this project is to design and develop a robust, intelligent, and user-friendly adaptive learning platform that personalises the educational experience for each student.

Specific Objectives:

- To design and develop a full-stack web application using HTML, CSS, JavaScript (frontend) and Python FastAPI (backend)
- To implement a normalised MySQL relational database schema supporting all platform entities including users, courses, modules, lessons, questions, quiz attempts, and learner profiles
- To develop a Bayesian-inspired adaptive engine that updates a student's skill level score after every quiz attempt based on score and question difficulty
- To build a dynamic quiz system that selects questions at difficulty levels 1 (easy), 2 (medium), or 3 (hard) based on the learner's current assessed skill level
- To implement a smart lesson recommendation engine that identifies weak topic areas and prioritises relevant lessons for review
- To integrate a gamification system featuring XP rewards per completed lesson, daily streak tracking, and five achievement badges
- To provide a real-time progress analytics dashboard displaying skill level, quiz history, weak and strong topics, and course completion percentages
- To implement secure JWT-based authentication and role-based access control for three user roles: student, instructor, and admin
- To develop an admin panel enabling instructors to create courses, modules, lessons, and quiz questions without requiring database access
- To ensure system security through HMAC-SHA256 password hashing and parameterised SQL queries preventing injection attacks

7. TOOLS / TECHNOLOGIES USED

The system is developed using a modern, industry-relevant technology stack that provides flexibility, performance, and scalability.

Frontend:

- HTML5 — Structuring all web pages and user interface components
- CSS3 — Custom design system with CSS variables, animations, and responsive layout
- JavaScript (Vanilla) — Client-side logic, API communication via Fetch API, and dynamic DOM manipulation
- Google Fonts (Syne + DM Sans) — Typography for a modern, professional aesthetic

Backend:

- Python 3.10 — Primary programming language for backend development
- FastAPI — High-performance asynchronous web framework for building RESTful APIs
- pymysql — Pure Python MySQL client for executing raw SQL queries
- python-jose — JWT token generation and verification for authentication
- HMAC-SHA256 — Password hashing using Python's built-in hmac and hashlib libraries
- python-dotenv — Environment variable management for secure configuration

Database:

- MySQL 8.0 — Relational database management system storing all platform data
- 11 normalised tables with foreign key constraints ensuring referential integrity
- JSON columns for flexible storage of quiz options, answers, and topic arrays

Development Tools:

- Visual Studio Code — Primary IDE for all development tasks
- Uvicorn — ASGI server for running the FastAPI application
- MySQL Workbench — Database administration and schema visualisation
- Live Server (VS Code Extension) — Local development server for frontend files
- Swagger UI (FastAPI /api/docs) — Interactive API documentation and testing

8. SYSTEM ARCHITECTURE

The system follows a three-tier client-server architecture, cleanly separating the application into presentation, application, and data layers for maintainability and scalability.

1. Presentation Layer (Frontend)

The frontend consists of six HTML pages styled with a unified CSS design system and powered by vanilla JavaScript. The Fetch API is used to communicate with the backend REST API. Pages include: Login/Register, Dashboard, Course Catalog, Course/Lesson Viewer, Adaptive Quiz, Progress Analytics, and Admin Panel.

2. Application Layer (Backend — FastAPI)

The backend is built with Python FastAPI and exposes a RESTful API consumed by the frontend. It handles authentication, business logic, adaptive algorithm execution, and all database operations via raw SQL through pymysql. Key routers include: /api/auth, /api/courses, /api/quiz, and /api/progress.

3. Database Layer (MySQL)

MySQL stores all persistent data in 11 normalised tables. The learner_profiles table is the core of the adaptive engine, storing each student's skill_level, total_xp, streak_days, weak_topics, and strong_topics.

Adaptive Engine Flow:

```
Student logs in →  Learner profile loaded (skill_level retrieved) →
Quiz requested for a lesson →  Adaptive engine selects question
difficulty (1/2/3) based on skill_level →  Student submits answers →
Engine grades responses and calculates score →  skill_level updated
using Bayesian-inspired formula:      new_skill = current_skill +
(score − 0.5) × 0.15 × difficulty_weight →  Weak/strong topics
updated based on score threshold (< 60% = weak) →  XP awarded and
streak updated →  Badge eligibility checked and awarded →  Next
lesson recommended (prioritising weak topics) →  Results returned to
frontend
```

Database Schema Overview:

- users — id, name, email, password_hash, role, avatar_url, created_at
- courses — id, title, description, difficulty, instructor_id, is_published
- modules — id, course_id, title, order_num
- lessons — id, module_id, title, content, video_url, xp_reward
- enrollments — id, user_id, course_id, enrolled_at
- lesson_progress — id, user_id, lesson_id, completed, time_spent_s
- questions — id, lesson_id, question, type, options (JSON), correct, difficulty
- quiz_attempts — id, user_id, lesson_id, score, total_q, correct_q, answers (JSON)

- learner_profiles — id, user_id, skill_level, weak_topics, strong_topics, total_xp, streak_days
- badges — id, name, description, icon, condition_text
- user_badges — id, user_id, badge_id, earned_at

9. METHODOLOGY

The development of the Adaptive Learning Platform followed a structured, iterative approach ensuring all features were implemented, tested, and integrated systematically.

Step 1: Requirements Analysis

Identified the core requirements: adaptive quizzes, role-based access, gamification, progress analytics, and a full course management system. Defined three user roles (student, instructor, admin) and their respective permissions.

Step 2: Database Design

Designed a normalised MySQL schema with 11 tables. Established foreign key relationships ensuring referential integrity. Used JSON columns for flexible storage of quiz options and topic arrays. Created the learner_profiles table as the central data store for the adaptive engine.

Step 3: Backend Development (FastAPI)

Implemented RESTful API endpoints using Python FastAPI with raw SQL queries via pymysql. Developed JWT-based authentication with HMAC-SHA256 password hashing. Built four API routers covering authentication, courses, quizzes, and progress tracking.

Step 4: Adaptive Engine Implementation

Implemented the Bayesian-inspired Knowledge Tracing algorithm in adaptive.py. The engine updates skill_level after every quiz using the formula: new_skill = clamp(current + (score - 0.5) × 0.15 × diff_weight, 0, 1). Implemented weak/strong topic detection with a 60% score threshold and next-lesson recommendation logic.

Step 5: Frontend Development

Built six HTML pages with a cohesive dark space-themed CSS design system. Implemented a JavaScript API client (api.js) centralising all Fetch API calls. Developed interactive components including the adaptive quiz timer, skill ring canvas visualisation, and animated results screen.

Step 6: Gamification System

Implemented XP reward distribution per completed lesson and quiz attempt. Built daily streak tracking comparing last_active date to current date. Developed five achievement badges with automated eligibility checking after each quiz submission.

Step 7: Admin Panel

Built a three-tab admin interface allowing instructors to create courses, add modules and lessons, and add quiz questions at three difficulty levels — all without requiring direct database access.

Step 8: Testing and Debugging

Conducted end-to-end testing of all user flows including registration, login, course enrolment, lesson completion, quiz submission, and badge awarding. Resolved database connection issues, CORS configuration errors, and password hashing compatibility problems during the testing phase.

10. CONCLUSION

The Adaptive Learning Platform successfully demonstrates how modern web technologies and relational database management systems can be combined to deliver an intelligent, personalised, and engaging online learning experience.

The platform achieves its core objective of adaptive learning through the implementation of a Bayesian-inspired Knowledge Tracing engine that continuously evaluates each student's skill level and adjusts quiz difficulty accordingly. Students who perform well are challenged with harder questions, while those who struggle receive easier content and targeted recommendations for their weak areas.

The gamification system — comprising XP rewards, daily streaks, and achievement badges — significantly enhances student motivation and engagement, addressing one of the most critical challenges in online education: learner retention.

The technical implementation demonstrates proficiency across the full technology stack: a clean, responsive frontend built with HTML, CSS, and JavaScript; a robust RESTful backend built with Python FastAPI; and a well-normalised MySQL database schema supporting all platform operations with referential integrity.

The role-based access control system, JWT authentication, and HMAC-SHA256 password hashing ensure that the platform meets basic security requirements appropriate for an educational application.

Overall, this project provides a strong foundation for further development. Future enhancements could include video lesson support, email notifications, a leaderboard system, certificate generation on course completion, and a more sophisticated adaptive algorithm incorporating spaced repetition and forgetting curves.

The project has been a valuable learning experience encompassing database design, backend API development, frontend engineering, algorithm implementation, and full-stack integration — all core competencies of a professional software engineer.

11. REFERENCES

1. MySQL Documentation — https://dev.mysql.com/doc/

2. FastAPI Documentation — https://fastapi.tiangolo.com/

3. pymysql Documentation — https://pymysql.readthedocs.io/

4. Corbett, A.T. & Anderson, J.R. (1994). Knowledge Tracing: Modeling the Acquisition of Procedural Knowledge. User Modeling and User-Adapted Interaction, 4(4), 253–278.

5. W3Schools — https://www.w3schools.com/

6. GeeksforGeeks — https://www.geeksforgeeks.org/

7. MDN Web Docs — https://developer.mozilla.org/

8. python-jose Documentation — https://python-jose.readthedocs.io/

9. Van Lehn, K. (2011). The Relative Effectiveness of Human Tutoring, Intelligent Tutoring Systems, and Other Tutoring Systems. Educational Psychologist, 46(4), 197–221.